

Challenge 3: Live-Data Engine

What you will build

You will build a scalable, resource-efficient system that keeps directory data reliably up to date — transforming static listings into a continuously maintained, trustworthy data stream.

The goal is not to display data.

The goal is to maintain data quality at scale, automatically.

Context

Directory data decays constantly. Businesses change their hours, update their services, move locations, or close entirely — without notifying anyone.

Directories that rely on manual updates fall behind quickly, and once the data is stale, the whole system loses credibility.

The challenge isn't detecting a single change. The challenge is building a system that can do this reliably, continuously, and at a scale where tens or hundreds of thousands of records are processed every month.

That scale constraint is the core design challenge of this problem. At that volume, expensive tools become infeasible by default. Calling a paid API for every record, or running a heavyweight model on every page crawl, simply doesn't survive contact with real numbers. The systems that actually work at scale tend to be leaner, smarter, and more deliberate about when and how they use expensive operations.

Your challenge

1. Detect changes automatically

Your system needs to know when something has changed — without being told. That means monitoring sources, identifying signals that indicate an update, and filtering out noise. Not every change to a webpage is a meaningful change to the listing.

2. Extract structured data

Once a change is detected, the relevant information needs to be extracted and mapped back to the directory's data model. Source content is messy,

inconsistent, and unstructured. Your extraction logic needs to handle that gracefully.

3. Update efficiently

Updates should be automated, accurate, and minimal. Overwriting correct data with incorrect data is worse than not updating at all. Think about confidence thresholds, validation logic, and how you handle uncertainty.

4. Scale realistically

This is where most demo-quality solutions break. Design your system as if it will process hundreds of thousands of records per month. That means being deliberate about:

- What runs on every record vs. what runs only when triggered
- What can be handled with deterministic logic (regex, rule-based parsing) vs. what actually requires a model
- What the cost and latency profile looks like at 10x and 100x current load

A system that works for 100 records but requires a paid API call per record does not work at 100,000. That's not a scaling optimization — it's a different architecture.

What we are NOT looking for

- One-time scraping scripts presented as a pipeline
- Solutions that depend on expensive per-request API calls at scale
- Black-box AI where no one can explain what triggered an update or why
- Manual processes dressed up as automation

If your solution only works because each record gets a dedicated LLM call, it is not production-ready. That's a prototype assumption, not an infrastructure assumption.

If it doesn't scale → it doesn't work.

Where to start

Think about how data actually breaks in the real world:

"The business updated their website but nobody told the directory"

"The hours changed three months ago — but the listing still shows the old

ones"

"Half the fields are empty because no one ever filled them in"

"The data was correct once — now it's just stale"

Then ask: how does a system catch that — without a human in the loop?

Don't think in terms of scripts. Think in terms of pipelines:

- How does your system know when to check a source?
 - How does it decide whether what it found is a real change or noise?
 - How does it map unstructured content back to structured fields?
 - How does it handle cases where the extraction is uncertain?
 - What does a maintenance and monitoring story look like?
-

Technical hints (optional)

- Regex and rule-based extraction as a cost-efficient first pass
- Small, self-hosted open-source models for targeted extraction tasks
- Diff-based change detection
- Confidence scoring and threshold-based update logic
- Pipeline orchestration

The most elegant solutions here will use expensive operations sparingly and deliberately — not as the default path.

Resources & Setup

Available data

- SMB dataset (base directory with existing listings)
- Sample input sources: websites, text snippets, simulated updates

Constraints to keep in mind

Your solution will be evaluated partly on its resource efficiency. A solution that produces great results but would cost €10,000/month to run at scale is not considered a viable architecture — even if the output quality is high. You don't need to provide exact cost projections, but your design choices should reflect an awareness of what scales and what doesn't.

Optional environment

- Backend starter
- Pipeline examples

Focus on logic and architecture clarity. A well-explained simple pipeline beats an impressive-sounding black box.

Demo expectations

Show the full pipeline clearly:

1. **Input** — what data source is your system monitoring?
2. **Change detection** — how does it know something changed?
3. **Extraction logic** — how is the relevant information identified and structured?
4. **Update mechanism** — how does the directory entry get updated?
5. **Output** — what does the updated listing look like?

Make the system understandable. We want to see how decisions are made, what triggers an update, and how errors or uncertainty are handled. A clear and explainable pipeline is more valuable here than a technically impressive one that no one can follow.

This challenge does not require a polished frontend. But the system's behavior should be clearly demonstrable — through logs, visualizations, before/after comparisons, or whatever makes the logic legible.

Evaluation criteria

Criterion	Focus
Feasibility	Could this architecture realistically be built and maintained?
Efficiency	Is it resource-aware — does it avoid unnecessary cost at scale?
Accuracy	Does it produce correct, reliable updates?
Scalability	Does the design hold up at tens or hundreds of thousands of records/month?
Clarity	Can you explain how it works and why it makes the decisions it does?

One sentence

Turn outdated directory data into a living, self-updating system — built to run at scale without breaking the bank.